

CSIC30216

AI-in-the-Loop in Software Project Cycle

AI 導入軟體專案流程

Week 3: Working with AI in a Team

Instructors:

Dennis Wang 王邦任

Yu-Chun (Grace) Yen 顏羽君

Date: 2026-09-21

Announcements

(before we get started)

W3 Announcements

- Course ADD/DROP due date is **TODAY**
- Late ADD Students
 - Email us when submitting your lab/reading to receive full credit
- We will host a [Team Matching session](#) during today's lab
- Please fill out the [Course Start Survey](#) by the end of this week (Friday, September 25, 23:59)
 - You're required to finish the survey — you'll receive 1% of semester grade after finishing it
- **Please remember to finish weekly assignments** — about 1/5 of you have not finished your week 2 lab & self-paced module questions.

Learning Objectives

- Explain what distinguishes professional software engineering from a coding assignment and how version control exists specifically to make that reality survivable
- Elaborate the relationship between a team's process and whether that team can resolve disagreements and maintain the codebase under pressure later in the semester, rather than treating process as bureaucracy separate from the work
- Recognize how a tool's human-AI decision boundary shapes what "success" means for it, and what an evaluation plan needs to measure

Why this week comes before the project

- Teams that establish AI usage expectations before building avoid the most common failure modes
- Teams that skip this step tend to: have undocumented prompts nobody else can maintain, have unreviewed AI commits that break things silently, have disagreements about AI output quality with no way to resolve them
- The norms you write today are not bureaucracy — they are the infrastructure your collaboration runs on

What today's lecture/lab covers

- First 30 minutes: the norms layer — how your team agrees to work with AI
- Second 30 minutes: the evaluation layer — how your team decides whether your tool actually works
- Lab (60 minutes): set up the repository, write the first drafts of both
 - It will just be a rough first draft, don't worry about it being very draft-y

Software Engineering In A Nutshell ***(aka why/how to version control)***

Software engineering in a nutshell

- Differentiating Programming and Software Engineering:
 - **Programming** is writing code that works right now.
 - **Software engineering** is building systems that keep working — over time, under changing requirements, with other people touching the same code.

How is SE in industry different from a coding assignments?

- work context is different from learning — stakeholders step in
 - *“My boss has no idea what they want”*
- testing, then testing, then testing, then testing,...
- code matters beyond the 16 weeks period...
- code needs to fit within a bigger infrastructure
- team collaboration is required.
 - things like *documentation, communication* needs to be done...

How is SE in industry different from a coding assignments?

- Requirements may be ambiguous, changing during development
- Scale is way, way larger
 - Requires different design skills, teamwork
 - Software must be changed after development is complete
 - Failure is more expensive
 - Business-critical, Safety-critical, Power-dynamic

Software engineering in a nutshell

What matters?

- **Maintainability:** can someone else understand and safely change this code later?
- **Collaboration:** can multiple people work on it without stepping on each other?
- **Traceability:** can you tell why a piece of code exists, and what happened before it?

Software engineering in a nutshell

What are the practices people do?

- **Version control:** tracking every change over time, enabling collaboration without overwriting each other's work
- **Code review:** a second person reads and approves changes before they merge
- **Automated testing and CI:** tests run automatically on every change to catch breakage early

→ *This is what today's lab is about!*

Software engineering in a nutshell

What are the practices people do?

- **Documentation:** design docs, decision records, comments — the reasoning survives past the person who had it
- **Issue and project tracking:** what needs to be done, who's doing it, what's blocked
- **Style guides and linters:** consistency enforced automatically rather than argued about in every review

Software engineering in a nutshell

What will we do in class?

- Most of what feels like "process" in this course exists to answer those three questions, not to add *bureaucracy*
- PR templates, DECISIONS.md, code review, branch protection
- *We'll talk about this in a bit!*

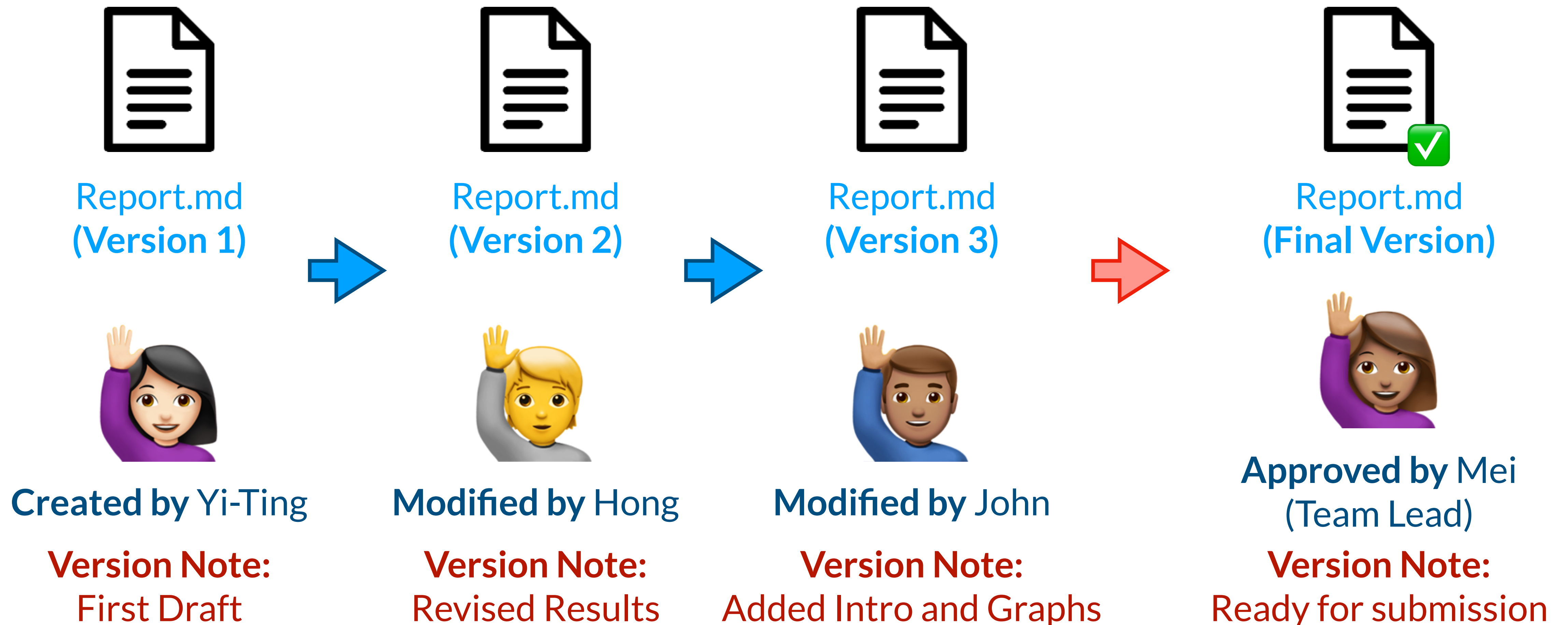
Software engineering in a nutshell

How does AI change the process?

- AI is changing how the process is done
 - AI-generated documentation — drafts of README files, docstrings, and design docs
 - AI-assisted code review — flagging likely issues before a human reviewer looks
 - AI-generated tests — drafting test cases for a function or module
 - AI-drafted PR descriptions and commit messages — a starting point a human still has to verify and take ownership of
- But "AI wrote it" doesn't change the standard:
 - The code still has to be *maintainable*, *collaborative*, and *traceable* regardless of who or what wrote it

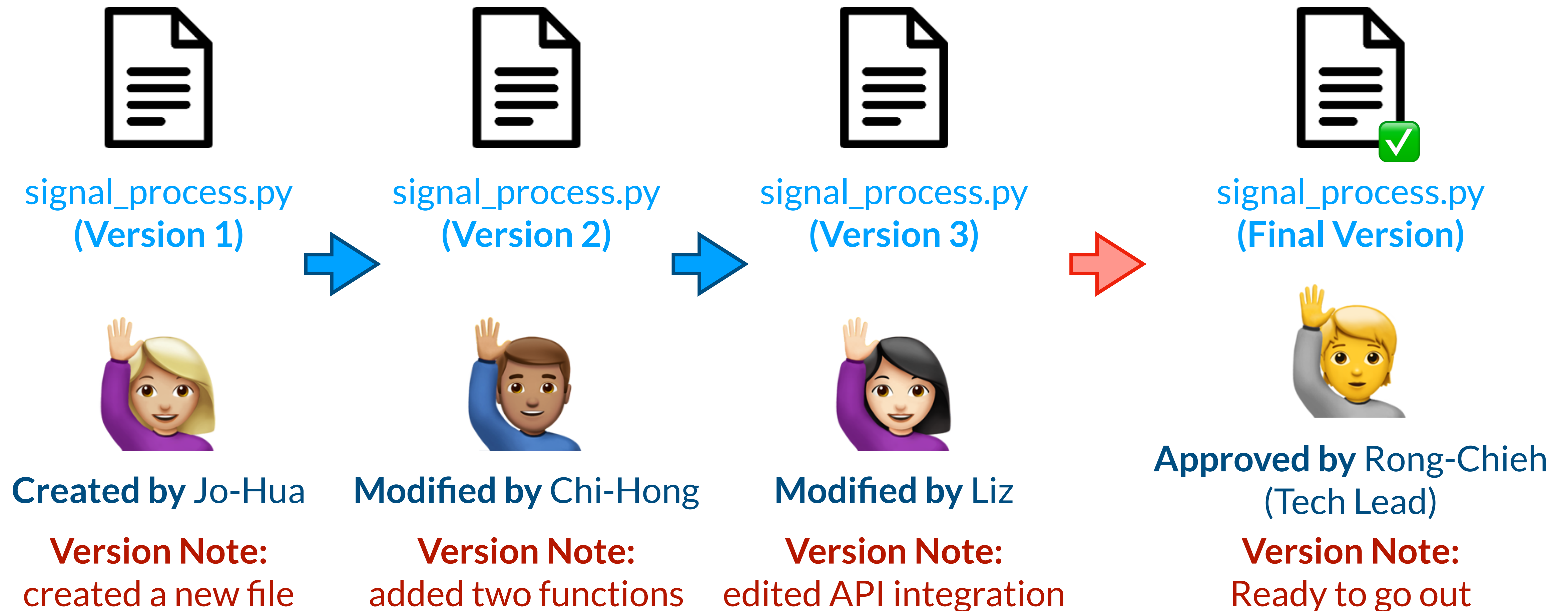
Version control

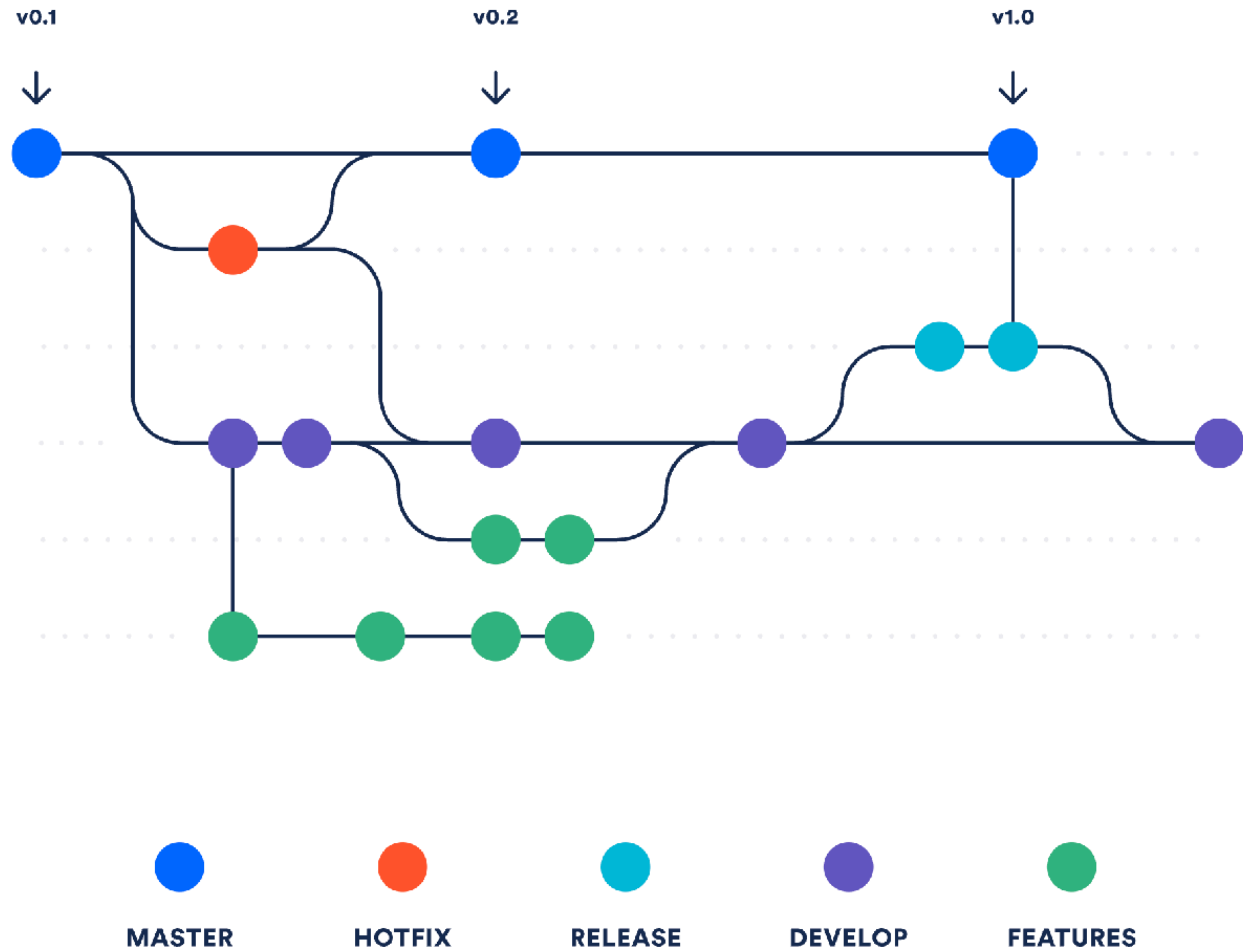
Working on a *document* together...



Version control

Working on a *file* together...





Version control: what git actually does

- The problem version control solves:
 - multiple people need to change the same code at the same time, without overwriting each other, while keeping a reliable history of what changed, when, and why — and a way to undo mistakes
- **Without it:** files named *final_v2_FINALFINAL.py*, changes silently overwritten by whoever saved last, no record of why a change was made
- **Git** is the current industry standard for solving this — it's what the overwhelming majority of professional software teams use day to day, this semester included
- Today's lab is your team's first hands-on use of it together, not just individually

Version control: what git actually does

Core ideas

- A **repository** is the full history of a project — every saved change, from the first commit to now
- A **commit** is a saved snapshot of the code at one point in time, with a message explaining what changed and why
- A **branch** is a parallel line of work — you can experiment or build a feature without touching the working version other people rely on
- A **pull request (PR)** is a proposal to merge one branch into another — it's where review happens before code becomes permanent

Version control: what git actually does

Core ideas

- Nothing in today's lab works without these four ideas:
 - branch protection protects the branch,
 - the PR template documents the commit's reasoning,
 - DECISIONS.md is a curated view of the most important commits

The AI-augmented PR norm

- Every PR description must explain:
 - what the AI produced and what you changed
- Not a paragraph — one or two sentences in the PR template
 - *"Claude generated the initial implementation of validate_input(). I modified the error handling to use our custom exception class and added the empty string edge case it missed."*
- This is what makes code review for AI-generated code possible — the reviewer knows what was human judgment and what was AI output

The AI-augmented PR norm

Commit **6174777**



dwnnis and **claude** committed last week

Add Week 2 self-paced module page

Adds pages/self-paced/week2.html covering in-context learning and instruction tuning (FLAN, RLHF, Constitutional AI), following the section structure of week1.html. Links it into the calendar via self_paced_url in _data/schedule.yml.

Co-Authored-By: Claude Sonnet 5 <noreply@anthropic.com>

Claude-Session: https://claude.ai/code/session_01A1WA26rSSfhviCaChUVQzW

*What working with Git in agentic development environment looks like
(Claude Code, Codex, etc.)*

Git workflow norms for AI-assisted teams

- **Atomic commits:** one concern per commit — AI-generated changes should be a separate commit from hand-written changes so they are traceable
- **Branch naming:** signal intent — feature/validate-input-ai-generated tells the reviewer something; update does not
- **One concern per PR:** when AI generates a large block of code, resist the urge to merge it all at once — split into reviewable units
- These are not new principles — they are standard git hygiene that matters more when AI is generating significant amounts of code

Handling disagreements about AI output quality

- **Question:** one engineer thinks the AI-generated code is good enough; another does not. How do you resolve it?
 - **The wrong answer:** whoever has more seniority wins
 - **The right answer:** evidence established in advance — what does "good enough" mean for this team on this project?

Handling disagreements about AI output quality

- **Example criteria:**
 - Passes the test suite
 - Passes the PR review checklist
 - Is understandable to someone who did not generate it
 - Does not introduce security issues flagged by the review

DECISIONS.md

A professional norm, not a course requirement

- A file in your repository root recording significant decisions: what was chosen, why, what alternative was rejected
- Format: two sentences per entry — one on what, one on why
- Not for routine changes — for decisions where the reasoning matters to a future engineer
- Real engineering teams use this (under various names: Architecture Decision Records, design docs, decision logs) — you are learning a professional practice, not inventing one

Disclosure and liability — briefly

- "The AI generated it" is not a legal or ethical defence
 - Company should be held liable for incorrect information the chatbot provided
 - The Air Canada chatbot case (2024) (which is covered in the Self-paced Module) — *"the chatbot is a separate entity" was rejected as a defense*
- If you deploy a system that harms users, you are responsible — not the model provider

The most common team AI failure mode

Imagine this...

- An engineer (you?) uses AI Model to generate a key function
- The prompt is never documented
- Three weeks later the function breaks
- Nobody knows what the prompt was, why that implementation was chosen, or what alternatives were rejected
- The team spends two hours reconstructing a decision that took five minutes to make

AI Usage Guideline

- A written, team-agreed answer to three questions:
 - Which AI tools we use for what
 - How AI-assisted work gets documented
 - How disagreements about AI output quality get resolved — not a vague statement of support for "responsible AI use"

AI Usage Guideline

- Common elements across published guidelines, regardless of company:
 - An *approved-tools* list
 - *Data Usage Rules*: what data can and can't be entered into AI tools
 - *Documentation requirements* for AI-assisted changes
 - *Review requirements* that scale with the risk of the code being touched
 - A named *owner responsible* for keeping the guideline current

What makes an AI usage guideline actually useful

- Specific enough that a new team member could follow it without asking questions
 - "Use AI responsibly" → **not useful**
 - *"Use Claude Haiku for generating test cases; use Claude Sonnet for code that will be merged to main; document every Sonnet interaction in the prompt engineering log with the prompt, the output, and what you changed"* → **useful**
- The test: read your guideline to someone who was not in the room when you wrote it. Can they follow it?

Three sections every team guideline needs

- **What tools, for what:** which AI tools the team will use and specifically what each is used for — not "we will use Claude" but "we will use Claude for X and Y, not Z"
- **How AI interactions are documented:** what goes in the prompt engineering log, what goes in DECISIONS.md, what goes in the PR description — concrete and specific
- **How disagreements about AI output quality are resolved:** if two engineers disagree about whether AI-generated code is good enough to merge, how is that resolved? Evidence, not opinion — but what evidence?

Evaluation (why we should test with real users)

From norms to evaluation

- You now know how to govern AI use in your team
- The next question: how do you know whether your tool actually works?
- This is not a question you answer at the end — it is a question you answer before you start building

Why evaluation thinking before building

- Teams that define success before building make different design decisions
- If you know your success metric is "users can complete the primary task without asking for help," you build a different tool than if you are optimising for "users say they like it"
- The evaluation plan is a design document — it shapes what you build, not just how you measure it

Output metrics vs outcome metrics

- **Output metric:** did the system produce the thing it was supposed to produce?
 - "The AI generated a summary" — output metric
 - "The PR review tool flagged at least one issue" — output metric
- **Outcome metric:** did the system change something that actually matters?
 - "Engineers spent less time re-explaining context at PR handoffs" — outcome metric
 - "Teams merged PRs with fewer back-and-forth comment cycles" — outcome metric
- Output metrics are necessary but not sufficient — a system that produces the right output but does not change anything real has not solved the problem

Output metrics vs outcome metrics

Two examples: output vs outcome

- Meeting summarizer
 - **Output metric:** produces a summary with action items
 - **Outcome metric:** team members who were absent from the meeting can make decisions based on the summary without asking follow-up questions
- PR review assistant
 - **Output metric:** generates review comments for every PR
 - **Outcome metric:** PRs that go through the AI review process require fewer human review cycles before merge

The human-AI decision boundary question

If your tool uses AI.....

- If your tool uses AI: what does the AI decide, and what does the human decide?
- This question must be answered before you start building — the answer should shape your interaction design
 - **High autonomy example:** AI reads the PR, generates a review, auto-posts it. Human is notified after the fact.
 - **Human-in-the-loop example:** AI generates a draft review. Human reads, edits, and posts it.
- These are different products. The first is faster; the second is more trustworthy and legally defensible.

Why this question matters for your evaluation plan

- If the AI makes decisions autonomously, your evaluation must measure whether those decisions are correct
- If the human is in the loop, your evaluation must measure whether the AI's suggestions are useful enough to be worth the interruption
- A tool where the AI decides everything and sometimes decides wrong is not the same evaluation problem as a tool where the human always decides with AI assistance

Three lightweight evaluation approaches

- **Structured observation:** watch two people use the tool together; record what they do, what confuses them, where they ask for help
- **Pre/post survey:** ask target users about the collaboration pain point before and after using the tool; measure the delta
- **Brief interview:** five questions, 15 minutes, one or two real users who are not your teammates
- Each approach has a different cost, different richness, and different appropriate use case — the self-paced module covers each in depth

The most common evaluation mistake

- Below is a list of things that you might do, which will not help you move forward
 - Testing only with your own team
 - Your team knows how the tool works, knows its quirks, is motivated to make it succeed, and is not representative of real users
 - A tool that works well for your team and fails for everyone else is a failed tool
- **Minimum viable external validation:** two people outside your team who have the collaboration problem you are solving

What a good evaluation plan looks like

- Walk through a completed example — [show a fictitious but realistic evaluation plan on screen]
- Success definition: specific, observable, not "users are happier"
- Target users: named type, not "users" — "junior engineers doing their first async PR reviews"
- Method: which approach, roughly when in the project
- Minimum evidence threshold: what would convince a skeptical reader?
- Human-AI boundary: if AI is involved, what does it decide vs what does the human decide?

Evaluation approaches in depth

- What does it actually look like to run this within a semester project?
- **Structured observation:** two people, a task, a note-taker — 30 minutes produces more data than a week of surveys
- **Pre/post survey:** five questions before the tool exists (about the pain point) and the same five questions after — the delta is your evidence
- **Brief interview:** not a usability test, not a focus group — five specific questions, recorded (with permission), transcribed

Lab 3:

Repository setup + team norms + evaluation plan

Lab for today

- You have 60 minutes
 - Three deliverables, all submitted as pull requests by the end of class
 - Part 1 (15 min): repository setup — TA-guided, whole class together
 - Part 2 (20 min): draft AI usage guidelines — three sections, submitted as PR
 - Part 3 (20 min): draft evaluation plan — five fields, submitted as PR

Lab Part 1: repository setup (TA-guided, 15 min)

- **Step 1:** one team member creates the repository; all others added as collaborators with write access
- **Step 2:** enable branch protection on main (TA walks room — do not move on until confirmed)
- **Step 3:** add the PR template (.github/PULL_REQUEST_TEMPLATE.md — copy from course portal)
- **Step 4:** create DECISIONS.md — first entry: who is the code steward and why
- **Step 5:** verify — one member tries to push directly to main. It should fail.

The PR template

What this PR does

[One sentence]

Decision (required if a significant choice was made)

Decision: [what was chosen]

Why: [what problem this solves or what alternative it beat]

Alternative rejected: [optional]

Led by: [name]

What counts as a significant decision

- Significant: choosing a technical approach when another was possible; changing the AI integration design; adding or removing a feature based on user feedback; any architectural choice that affects future code
- Not significant: fixing a bug; updating a label; adding a test case for an existing feature; formatting changes
- When in doubt: if a future engineer reading the git history would want to know why, write it down

Lab Part 2: draft AI usage guidelines (10 mins)

- Three sections to complete — template on the course portal
 - Section 1: what AI tools the team will use and what each is specifically used for
 - Section 2: how AI interactions will be documented — prompt log, DECISIONS.md, PR descriptions
 - Section 3: how disagreements about AI output quality will be resolved
- Submit as a PR — one person reviews and approves

Lab Part 3: draft evaluation plan (20 min)

- Problem grounding — template on the course portal
 - Section 1: Who specifically has the collaboration problem you are addressing
 - Section 2: What do they currently do instead of your tool?
 - Section 3: What would be observably different about their collaboration if your tool worked?
- Five fields — template on the course portal
 - Success definition: "We will know our tool works if..." — observable and measurable
 - Target users: who specifically, and how you will access them
 - Method: which evaluation approach and roughly when
 - Minimum evidence threshold: what would convince a skeptical reader?
 - Human-AI boundary: if using AI — what does AI decide vs human decide?
- Submit as a PR — code steward reviews and approves

Lab Part 3: draft evaluation plan (20 min)

- Problem grounding (10 min) — template on the course portal
 - Section 1: Who specifically has the collaboration problem you are addressing
 - Section 2: What do they currently do instead of your tool?
 - Section 3: What would be observably different about their collaboration if your tool worked?
- Evaluation plan draft (10 min)—
 - Success definition
 - Target users
 - Method
 - Minimum evidence threshold

Before the lab starts

- Sit with your team, if possible
- Select one person as the *code steward* of the day
 - What the *code steward* needs to do:
 - Approve the change made to the repository today
 - This will be an ongoing role in each team
 - Each person in a team needs to work as the code steward for at least a few times during the semester
 - You can negotiate who will be taking this role during the different phase of your project
- If you have not found a team, bring your phone / laptop / tablet and come to the front, and we'll try to figure something out

What the self-paced module covers this week

- Three evaluation approaches in depth: how to run each one within a semester project's constraints
- How real engineering teams adopt AI tooling: case studies, what worked, what created friction
- Disclosure and liability detail: the Air Canada case and what it means for engineers
- Reflection prompt: look at your draft guidelines after reading the case studies — are they specific enough?

What to do before next class (in two weeks)

- Complete the self-paced module (~60 min)
 - I recommend you to complete it this week while memory of today's lab is fresh
- Refine your team's guidelines and evaluation plan based on the self-paced module's reflection prompt
 - Have the repository infrastructure working before you leave today — if something is broken, fix it before Friday
 - Start identifying potential users outside your team for evaluation — you need at least two by Week 10
- Please fill out the [Course Start Survey](#) by the end of this week (Friday, September 25, 23:59)

Week 4 notice

- Week 4 is a national holiday — Teacher's Day (Sep 28)
 - No class, no lab, no graded work due
- Optional self-paced enrichment posted on E3 — one blog post and one Anthropic post
- We should have finalized the *Token support plan* by the time Week 5 class starts

Week 5 preview

- **Topic:** prompt engineering I – core techniques
- **Lab:** writing prompts well – code generation as the worked example
- **Bring your laptop, we will have tasks that require you to write and run codes**

The three things you set up today

- **Repository infrastructure:** branch protection, PR template, DECISIONS.md — the technical foundation your collaboration runs on
- **AI usage guidelines:** what your team agreed to before anyone built anything
- **Evaluation plan:** how your team will know whether you solved the right problem

One thing to remember from today

- Teams that define what "good enough" means before a disagreement arises resolve disagreements in minutes
- Teams that do not define it spend hours on what should be a checklist question
- The norms you wrote today are that definition

Learning Objectives

- ✓ Explain what distinguishes professional software engineering from a coding assignment and how version control exists specifically to make that reality survivable
- ✓ Elaborate the relationship between a team's process and whether that team can resolve disagreements and maintain the codebase under pressure later in the semester, rather than treating process as bureaucracy separate from the work
- ✓ Recognize how a tool's human-AI decision boundary shapes what "success" means for it, and what an evaluation plan needs to measure

CSIC30216

AI-in-the-Loop in Software Project Cycle

AI 導入軟體專案流程

Week 3: Working with AI in a Team

Instructors:

Dennis Wang 王邦任

Yu-Chun (Grace) Yen 顏羽君

Date: 2026-09-21